

AM234/BME234 Homework 2

Due: Wednesday May 13, 2026 at 11:59 pm

1. (35 pt) Suppose that we are given an i.i.d. dataset $\{(x_1, y_1), \dots, (x_n, y_n)\}$ where $x_i \in \mathbb{R}$ and $y_i \in \{0, 1\}$ for all i . We would like to train a logistic regression model

$$f(x; \beta_0, \beta_1) = \frac{\exp(\beta_0 + \beta_1 x)}{1 + \exp(\beta_0 + \beta_1 x)} \quad (1)$$

using these data. The logistic regression model is also commonly written as

$$f(x; \beta_0, \beta_1) = \frac{1}{1 + \exp(-\beta_0 - \beta_1 x)}. \quad (2)$$

In this problem, we will walk through how to choose the β_0 and β_1 parameters.

- (a) (1 pt) Prove that (1) and (2) are equivalent. *It will be easier to use equation (2) for the rest of the problem.*
- (b) (5 pt) The sigmoid function is

$$g(z) = \frac{1}{1 + e^{-z}}.$$

Prove that the derivative $g'(z) = g(z)[1 - g(z)]$. *Hint: use the quotient rule!*

- (c) (1 pt) Rewrite the logistic regression model in (2) in terms of the sigmoid function g .
- (d) (8 pt) To choose appropriate values for β_0 and β_1 , we perform maximum likelihood estimation. Recall that the likelihood refers to the probability of observing the data under the models. For logistic regression, the likelihood is equal to

$$L(\beta_0, \beta_1) = P(Y|X; \beta_0, \beta_1).$$

Prove that the log-likelihood can be written as

$$\log L(\beta_0, \beta_1) = \sum_{i=1}^n y_i \log [g(\beta_0 + \beta_1 x_i)] + (1 - y_i) \log [1 - g(\beta_0 + \beta_1 x_i)] \quad (3)$$

- (e) (1 pt) To perform maximum likelihood estimation, we choose values of β_0 and β_1 that maximize (3). Why is it ok to maximize the log-likelihood as opposed to the original likelihood?

- (f) (10 pt) In order to choose values of β_0 and β_1 that maximize (3), we perform gradient ascent. In this part, we calculate the necessary gradients for gradient ascent. Compute $\frac{\partial \log L(\beta_0, \beta_1)}{\partial \beta_0}$ and $\frac{\partial \log L(\beta_0, \beta_1)}{\partial \beta_1}$.

Hint: remember to use the chain rule! If you are unfamiliar with calculating gradients or gradient ascent, check out this slide deck: https://www.cs.toronto.edu/~rgrosse/courses/csc311_f20/slides/lec02.pdf.

- (g) (2 pt) Using the gradients you calculated above, write the gradient ascent algorithm to optimize β_0 and β_1 . In particular, write the equations that gradient ascent will use to perform parameter updates until convergence (assuming that α is the learning rate).
- (h) (5 pt) Assume that the data are perfectly linearly separable (i.e. there exist values for β_0 and β_1 such that the logistic regression model achieves 100% accuracy). Why might gradient ascent still not converge to stable values of β_0 and β_1 ?
- (i) (2 pt) Provide a solution to fix the above convergence issue and explain why it works.

2. (10 pt) Multiple hypothesis testing.

- (a) (5 pt) Implement `apply_bonferroni_correction()` in `problem_2.py`.
- (b) (5 pt) Implement `apply_benjamini_hochberg()` in `problem_2.py`.

3. (35 pt) In this problem, you will conduct a GWAS on a simulated binary trait using real genotype data from chromosome 22 of the 1000 Genomes Project.

- (a) (12 pt) Conduct a GWAS to calculate a p-value for each SNP in `chr22_subsampled_snps.vcf`. Your code should be written in `problem_3.py`.

We recommend that you follow these steps:

- Calculate a dosage matrix X where $X_{i,j} \in \{0, 1, 2\}$ represents the number of alternative alleles individual i contains at SNP j .
- For each SNP j :
 - Train a logistic regression model of form $f(X_{ij}) = \frac{\exp(\beta_0 + \beta_1 X_{ij})}{1 + \exp(\beta_0 + \beta_1 X_{ij})}$ to predict Y from X_j . We recommend using the `Logit` class from the `statsmodels` library. Maximum likelihood estimation will fail on those SNPs that have the same dosage in every individual. For those SNPs, assume that the p-value equals 1.0.
 - For all other SNPs, determine the p-value for the hypothesis $H_j : \beta_1 = 0$. The `statsmodels` library automatically computes p-values for you, so you should not need to calculate this yourself.
- After getting p-values for every SNP, implement functions to perform multiple testing correction - you can either use your code from Question 2 or use functions from `statsmodels` (`statsmodels.stats.multitest.multipletests`).

- (b) (5 pt) Draw a Manhattan plot that displays the $-\log_{10}(p)$ value for each SNP. Also include two horizontal lines indicating the p-value cutoffs required to control the family-wise error

rate (FWER) and false discovery rate (FDR) at 0.05 after Bonferroni and Benjamini-Hochberg corrections, respectively.

- (c) (2 pt) For how many SNPs is the null hypothesis rejected after the Bonferroni correction? What about after the Benjamini-Hochberg procedure?
- (d) (4 pt) There were only five causal SNPs chosen to influence the phenotype in the original simulation. Explain why you reported so many more SNPs in part (c) even after correcting for multiple testing.
- (e) (8 pt) Redo the GWAS in part (a), but this time include the top three principle components from the dosage matrix as covariates. (We recommend using the `scikit-learn` library to run PCA. Please make sure to standardize the dosage matrix to have mean 0 and variance 1 along each column before running PCA). Again, draw a Manhattan plot that displays the $-\log_{10}(p)$ value for each SNP. How many SNPs are now reported as significant after Bonferroni correction and the Benjamini-Hochberg correction?
- (f) (4 pt) Why does including the top principle components reduce the number of significant SNPs and therefore make it easier to identify the causal SNPs?